Implementierung eines Caches in SystemC

Eren Önel, Rodrigo Martin Garcia, Miron Förster

Index

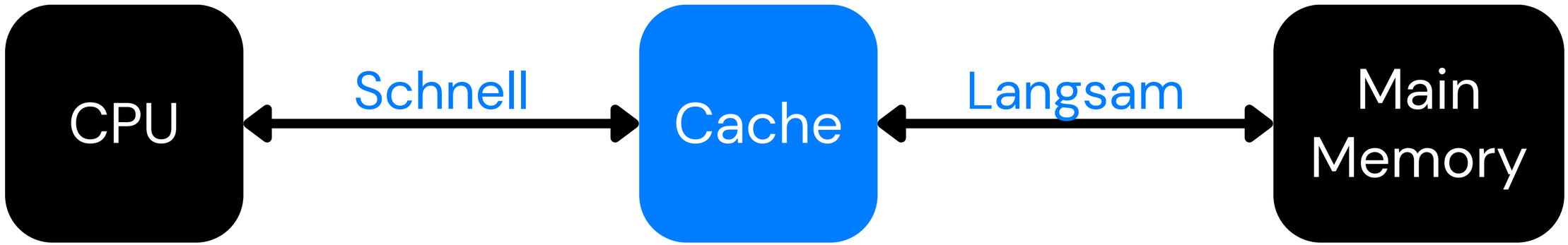
1 | Cache: Funktionsweise
2 | Aufgabenstellung
3 | Implementierung
4 | Simulation: Beispiele
5 | Zusammenfassung u. Reflexion

Cache: Funktionsweise

Cache: Funktionsweise



Cache: Funktionsweise

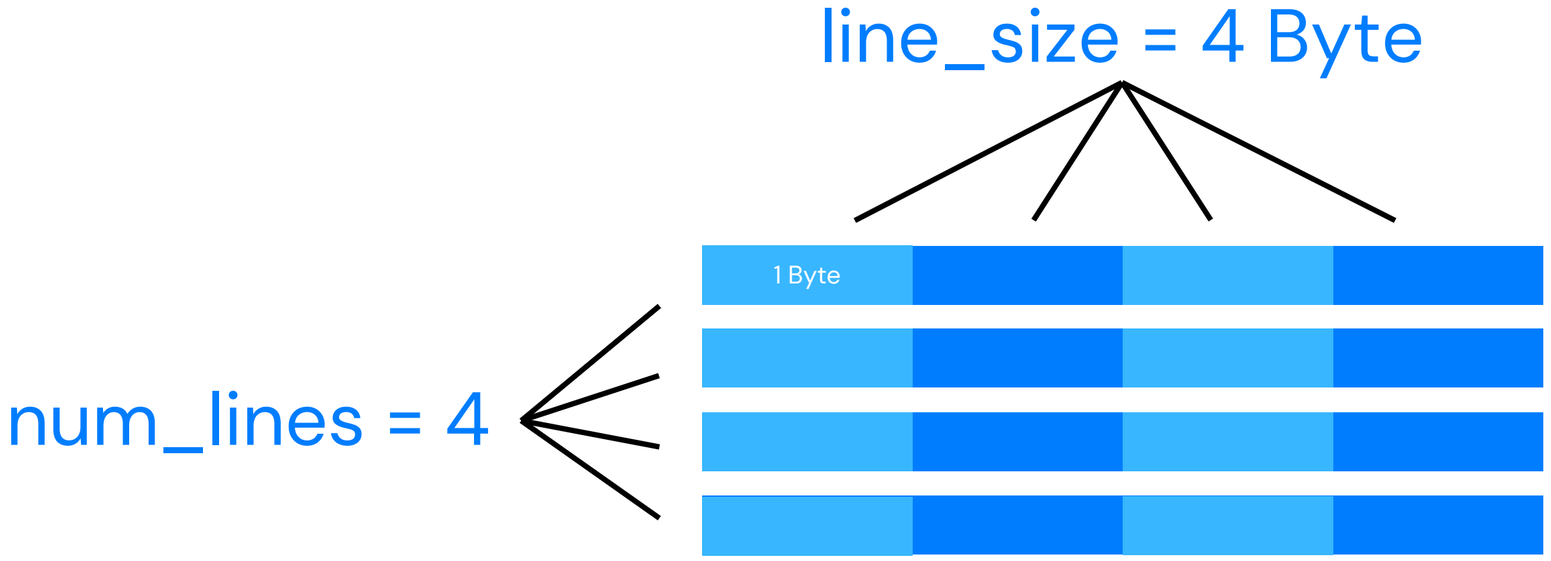


Kann Daten zwischenspeichern

→ Erneute Zugriffe auf selbe Daten sind schneller

Cache: Funktionsweise

Aufbau



Daten werden immer Zeilenweise in den Cache geladen

Cache: Funktionsweise

Mapping-Strategieen

Abhängig von der Adresse des Datums:
In welcher Cachezeile darf ein bestimmtes Datum abgelegt werden?

Direct-Mapped



Fully Associative



Cache: Funktionsweise

Replacement-Strategieen

Alle Zeilen, an denen ein Datum abgelegt werden
darf (nach Mapping-Strat) sind belegt:
Welche der Zeilen soll ersetzt werden?

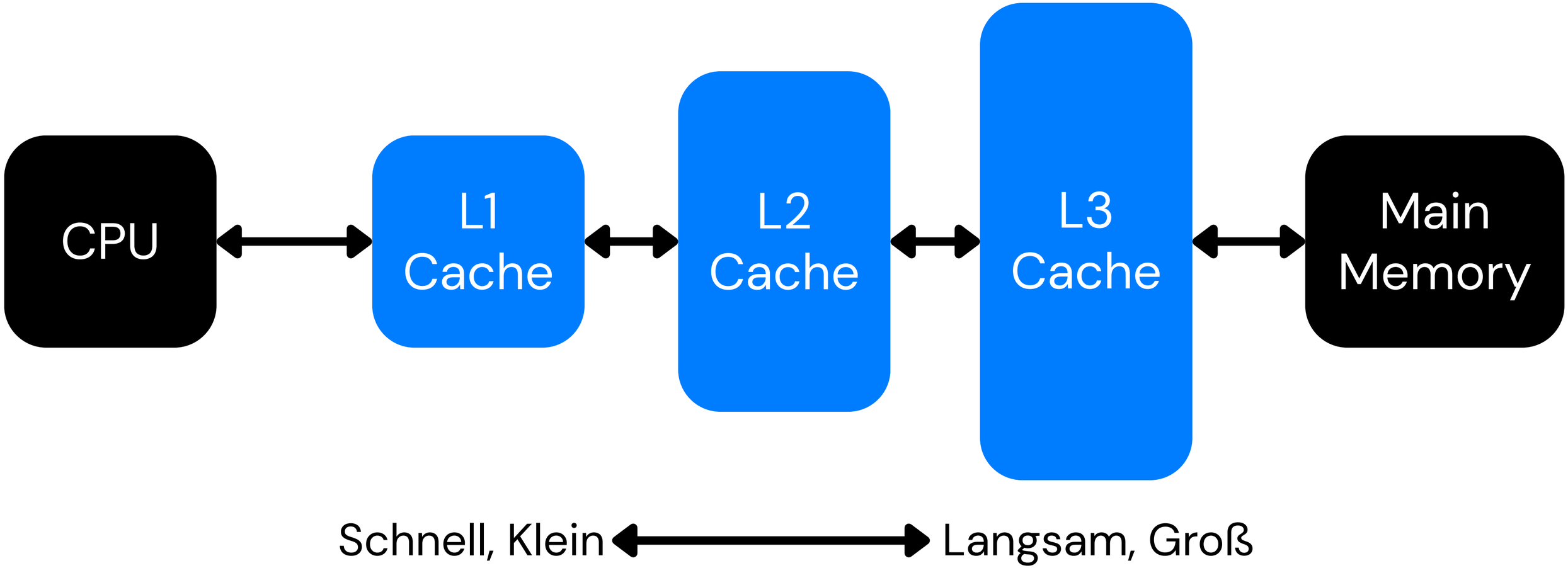
Cache: Funktionsweise

Replacement-Strategieen

- LRU (Least Recently Used)
- LFU (Least Frequently Used)
- FIFO (First In First Out)
- Random

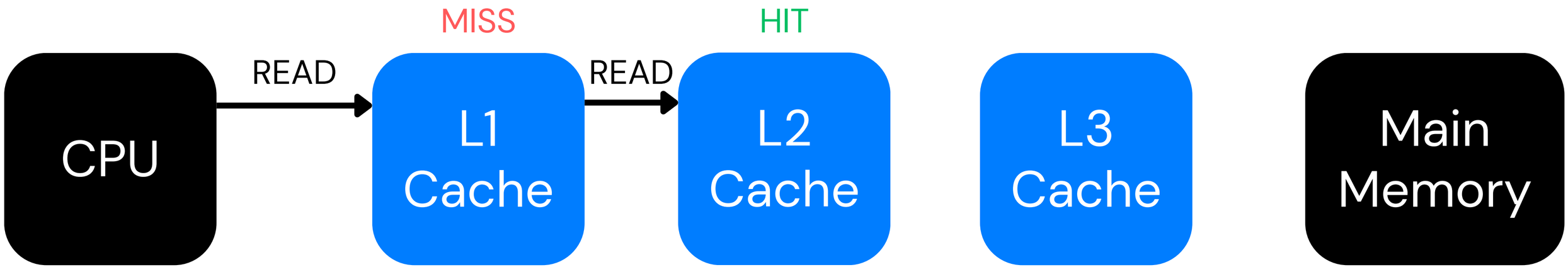
Cache: Funktionsweise

Multi-Layer-Cache



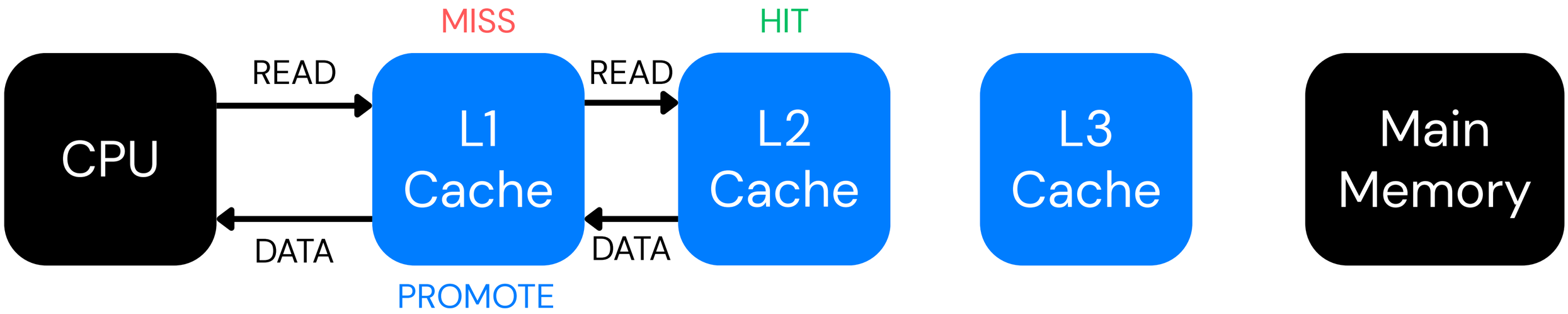
Cache: Funktionsweise

Prozess: Read-Hit



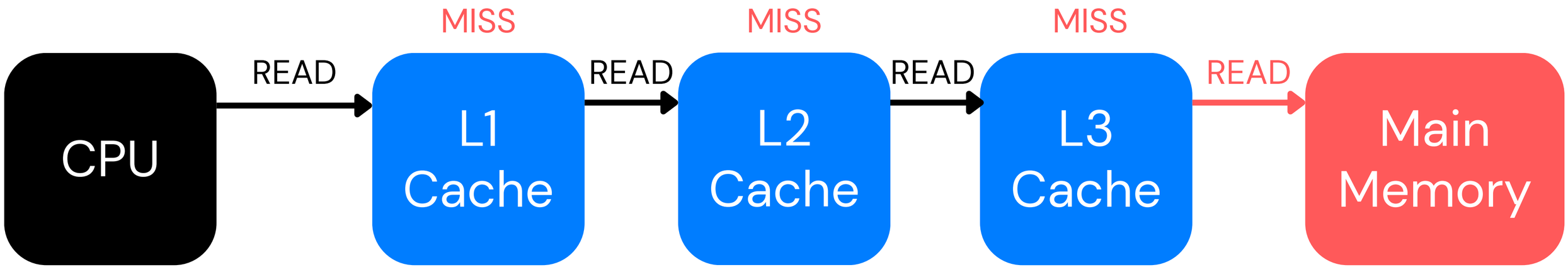
Cache: Funktionsweise

Prozess: Read-Hit



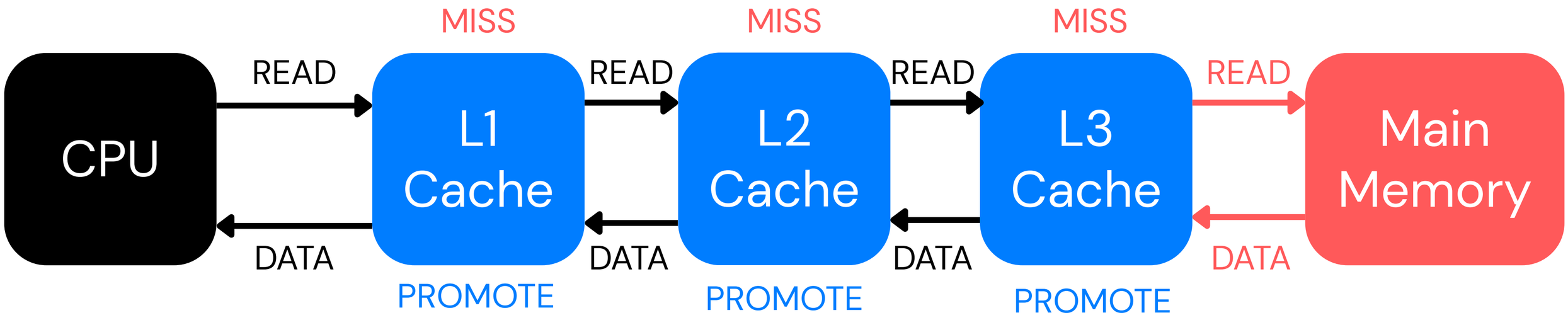
Cache: Funktionsweise

Prozess: Read-Miss



Cache: Funktionsweise

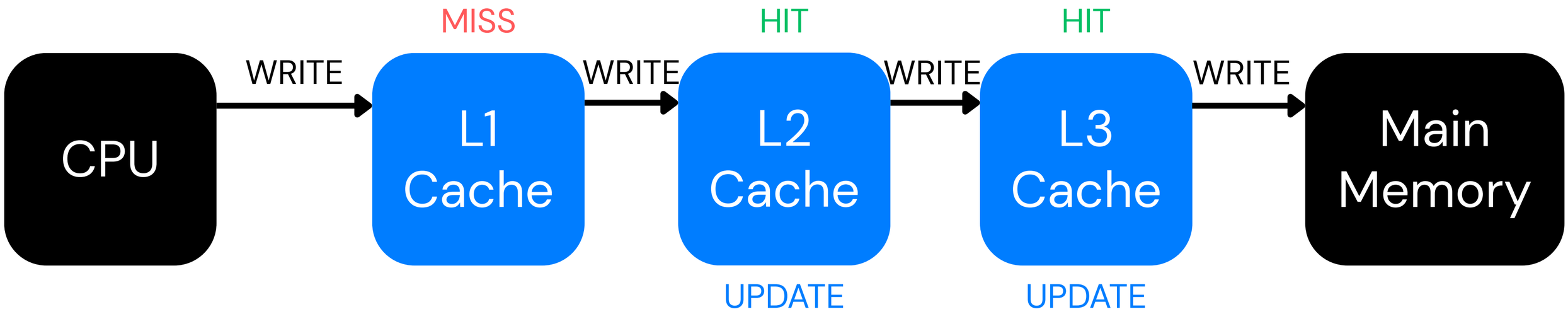
Prozess: Read-Miss



Cache: Funktionsweise

Prozess: Write-Hit

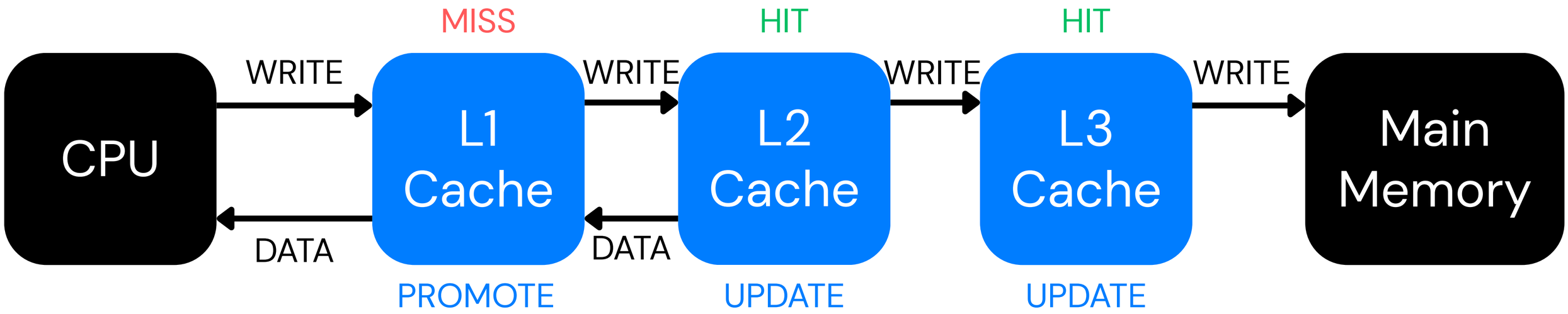
(Writethrough)



Cache: Funktionsweise

Prozess: Write-Hit

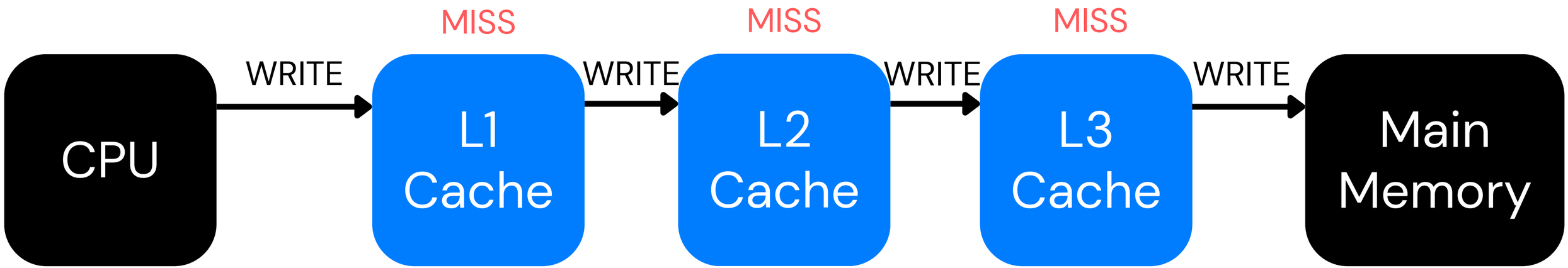
(Writethrough)



Cache: Funktionsweise

Prozess: Write-Miss

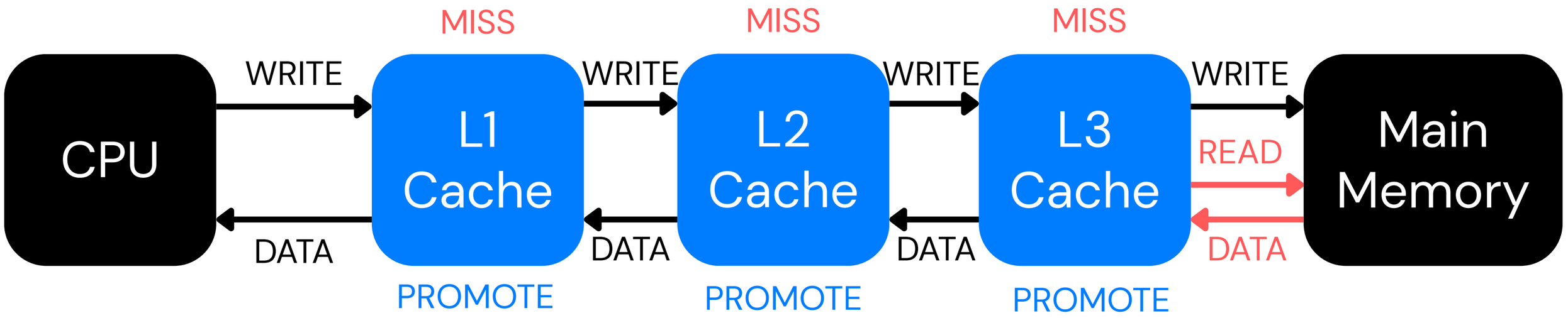
(Writethrough)



Cache: Funktionsweise

Prozess: Write-Miss

(Writethrough)



Aufgabenstellung

- Cache Modul
  - Byte-Addressierbar, immer 4-Byte-Reads/-Writes
  - Little Endian
  - Inklusiv
  - Writethrough
  - (Cacheline Promotion)
- Konfigurierbare Commandline Parameter
  - layers (1-3), Lines, Line sizes and latency
  - Mapping-Strategies:
    - Fully-Associative
    - Direct-Mapped
- Zugriffe aus CSV-Datei lesen und simulieren

Aufgabenstellung

Main Challenges

- Über die Requests aus der CSV-Datei iterieren
- Requests durch Layers durchreichen, Promotes zurückreichen
- Splitten von Requests in Subrequests wenn 4-Byte bereich mehrere Cachelines überlappt
- Mapping Strategieen korrekt implementieren
- Lesen einer Cachezeile aus Main-Memory in 4-Byte-chunks

Implementierung

Commandline-Parsing

number_converter

CSV-Datei Einlesen

run_simulation() Funktion

CACHE-Modul

LAYER-Modul

Implementierung

Commandline-Parsing

-c, --cycles <uint32_t>
-t, --tf <file>
-n, --num-cache-levels <uint8_t>
-s, --cacheline-size <uint32_t>
-a, --num-lines-l1 <uint32_t>
-b, --num-lines-l2 <uint32_t>
-d, --num-lines-l3 <uint32_t>
-x, --latency-cache-l1 <uint32_t>
-y, --latency-cache-l2 <uint32_t>
-z, --latency-cache-l3 <uint32_t>
-m, --mapping-strategy <uint8_t>
-h, --help

Implementierung

Number_converter

- Kann zwischen hexadezimal und dezimal Zahlen unterscheiden
- Zwei Methoden
  - Default: prüft auf Overflow mit 32 bit
  - Mit Maximum: prüft auf Overflow mit angegebenem Max (Bitanzahl)
- Erlaubt "_" zum trennen von Ziffern
  - Nicht am Anfang oder Ende
  - Nicht mehrfach hintereinander
- In Hexadezimal
  - Groß- und Kleinbuchstaben können beliebig gewählt werden (0x und 0X)
- Beispiel:
  - Erlaubte Zahlen: "0", "0xaB", "1_00", "0XDEAD_beef"
  - Nicht erlaubt Zahlen: "", "_10_", "12__9", "0x12_A", "0x_12"

Implementierung

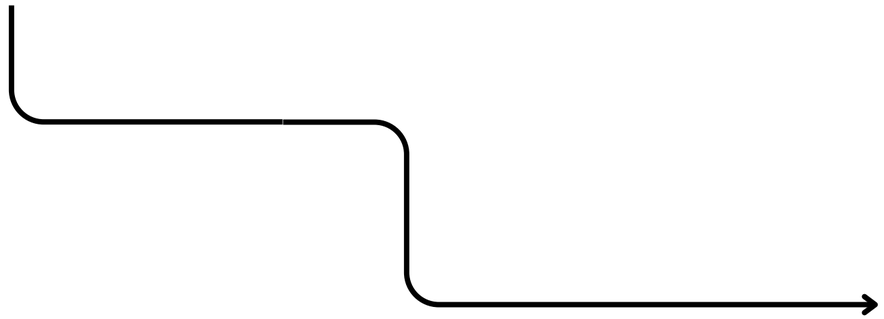
CSV-Datei Einlesen

- Zwei wichtige Methoden:
  - get_request_from_csv_file
  - get_request_from_line

Implementierung

get_request_from_line

"W,0xabc,0xabc" und "R,0xabc,"
w , 0xabc , 00001234
W , 000000000_1_2_2_3_443 , 1_2_3_4



```
typedef struct Request{
    uint32_t addr;
    uint32_t data;
    uint8_t w;
} Request;
```

Problem: Requests allokieren

- Drei verschiedene Methoden:
  - Zeilen zählen
  - Raten durch Charakteranzahl
  - Dynamisches allokieren

Implementierung

Zeilen zählen

- Pro:
  - Exakt -> keineReallokation
- Contra:
  - Große Datein brauchen länger
  - Verschiedene Zeilenbrüche
  - Kann leere Zeilen nicht erkennen

```
W,5384188,0x54e
R,718094,
W,5,0xafaea24
W,7606,6
W,0xcd38e,767645
R,0x7f4,
W,0xe75e,4174
R,506,
W,0x598dd28b,515221
W,45546368,0x3e1e4576
R,0xcee4,
W,0x977f11,231064
R,537714,
W,0x3c9f808,193425623
W,0x523f,0x875496
R,0xdfd57f3,
W,0xcc,31
W,49042019,7983
R,609,
R,0x82cfb5,
```

Implementierung

Raten durch Charakteranzahl

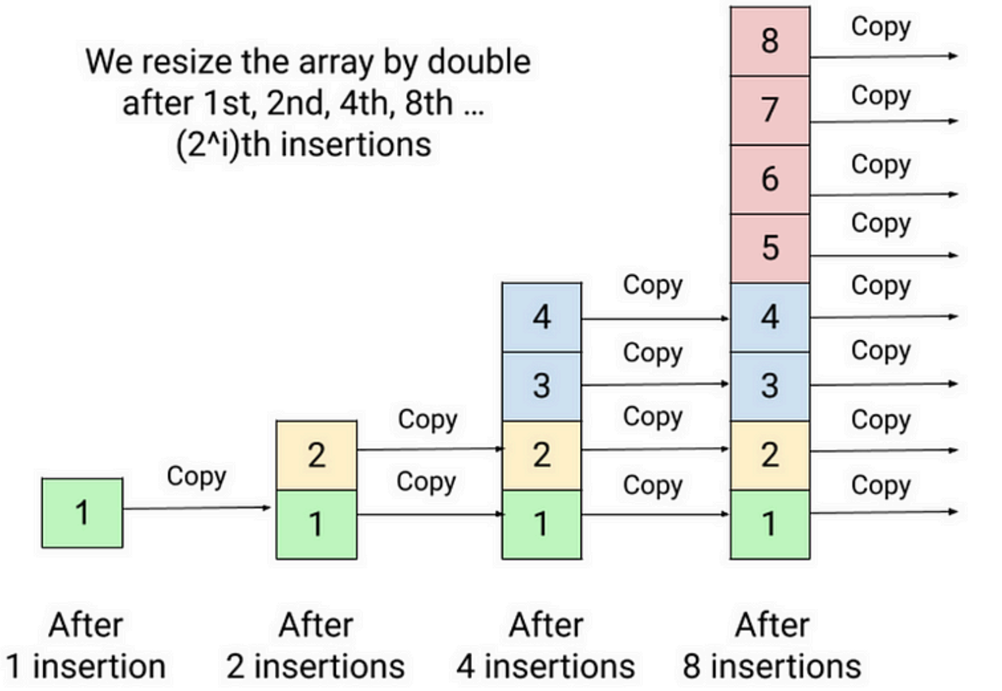
- Pro:
  - Kann sehr exakt sein
  - Gewöhnlich wenigerAllokationen
- Contra:
  - Anzahl an Charakters proZeile sehr variable
  - CPU intensiv -> vieleRechnungen

W , 936464966,6896
R,1,

Implementierung

Dynamisches Allokieren

- Pro:
  - Amortiesiert: O(1)
  - Muss nicht durch die csv-datei
- Contra:
  - Bei großen Dateienspeicherineffizient
  - Log2(n) Allokationen



https://miro.medium.com/v2/resize:fit:1200/0*j5_PlpMK8LrRjFAt.png

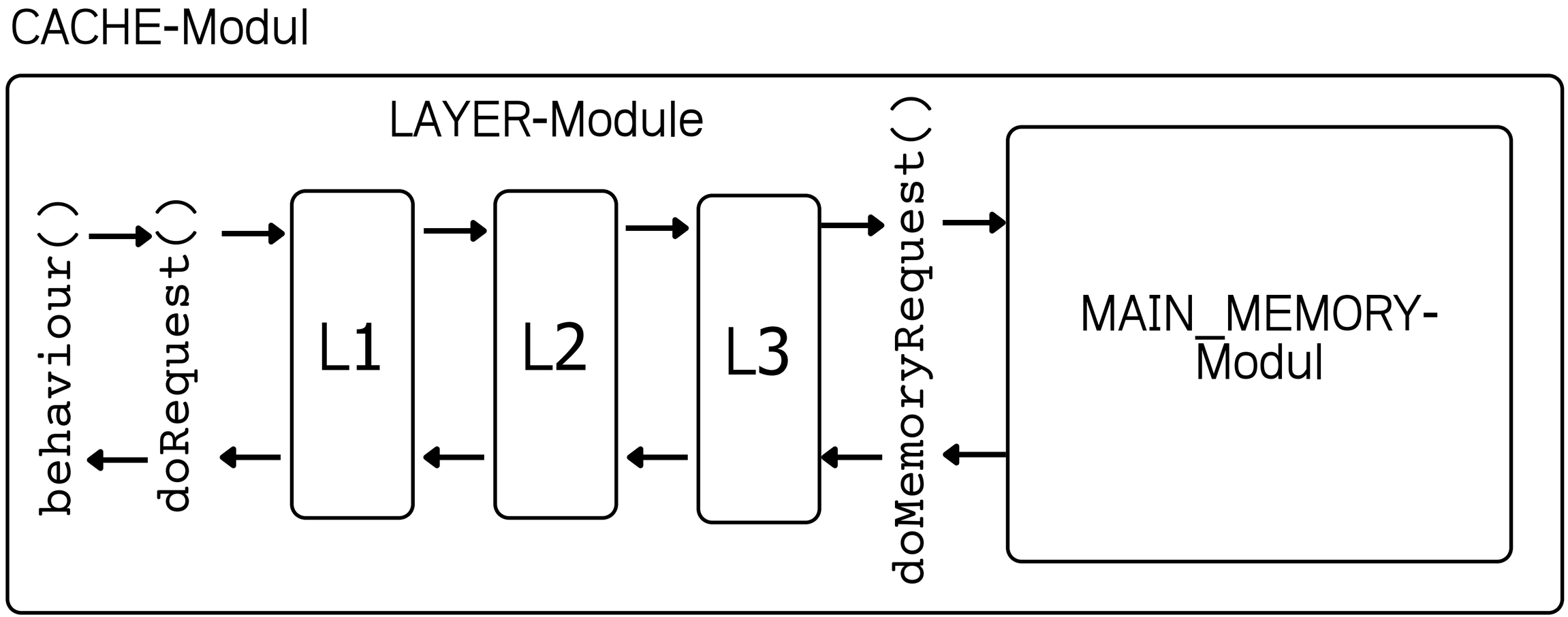
Implementierung

run_simulation() Funktion

- nimmt Argumente vom C-Teil entgegen
- Commandline-Logging
- Tracefile einrichten
- Über Requests iterieren, schreiben der Werte an Cache-Inputs
- Result-Statistiken Sammeln

Implementierung

CACHE-Modul



Implementierung

CACHE-Modul Threads

```
SC_THREAD(behaviour);
sensitive << clk.pos();
dont_initialize();

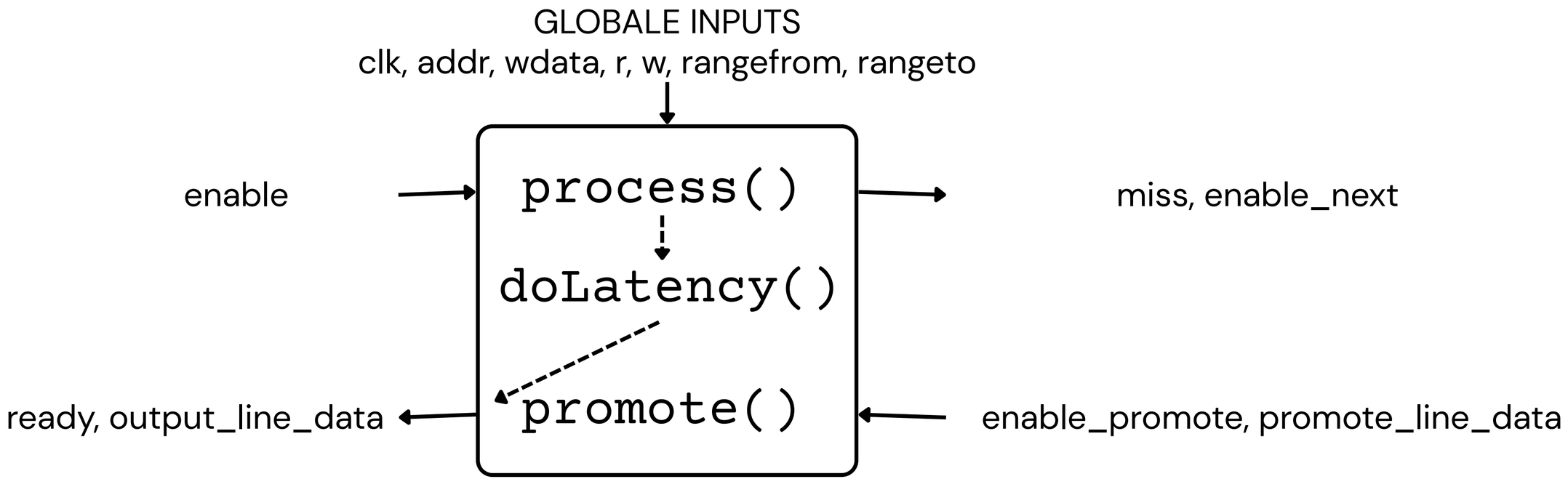
SC_THREAD(doRequest);
sensitive << internal_ready;
dont_initialize();

SC_THREAD(doMemoryRequest);
sensitive << memory_request_enable;
dont_initialize();

SC_THREAD(writeSignals_OnEvents);
```

Implementierung

LAYER-Modul



Implementierung

LAYER-Modul Threads

```
SC_THREAD(writeReady_OnEvents); // workaround for writing ready from different threads

SC_THREAD(writeStoredLineToOutputLine_OnEvent); // workaround for writing output_line_data from different threads

SC_THREAD(doLatency);
sensitive << clk.pos();
dont_initialize();

SC_THREAD(process);
sensitive << enable;
dont_initialize();

SC_THREAD(promote);
sensitive << enable_promote;
dont_initialize();
```

Simulation: Beispiel

Simulation: Beispiel

Beispiel 1: Matrix-Multiplikation

Ohne cache:

12 Write zugriffe

16 Read zugriffe

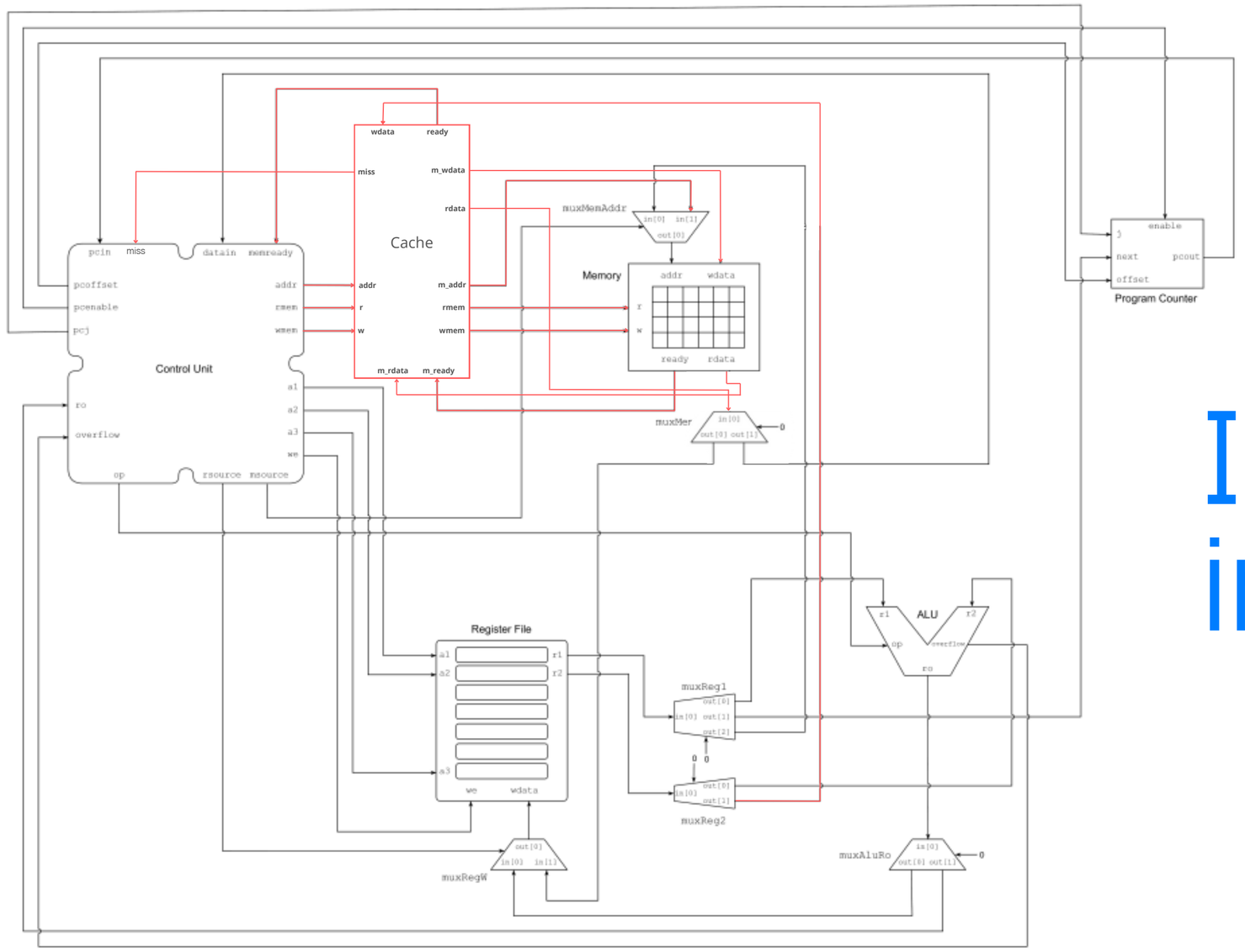
mem latency = 100 cycles

(12+16)*100 = 2800 cycles

Mit cache: 2460 cycles

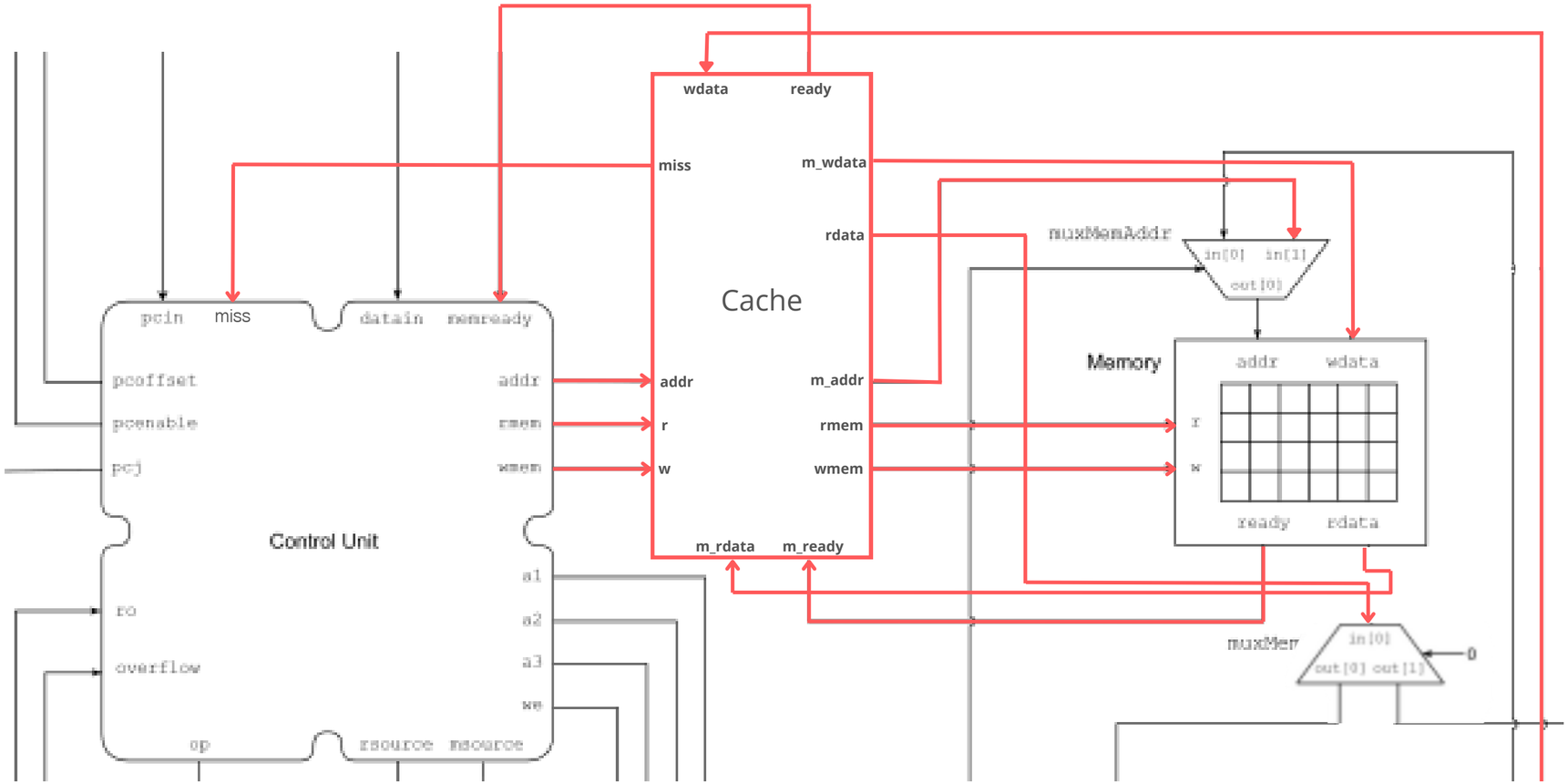
ca 15% improvement

Zusammenfassung



Integration in TinyRisc

Integration in TinyRisc



Zusammenfassung

Verbesserungspotenzial

- Mehr Mapping-Strategies
- More complex replacement-policies
- Writeback
- Prefetching
- Mehr Events statt Signals als Process Trigger
- Iterieren über die Requests innerhalb der Simulation

Danke für eure Aufmerksamkeit

Eren Önel, Rodrigo Martin Garcia, Miron Förster